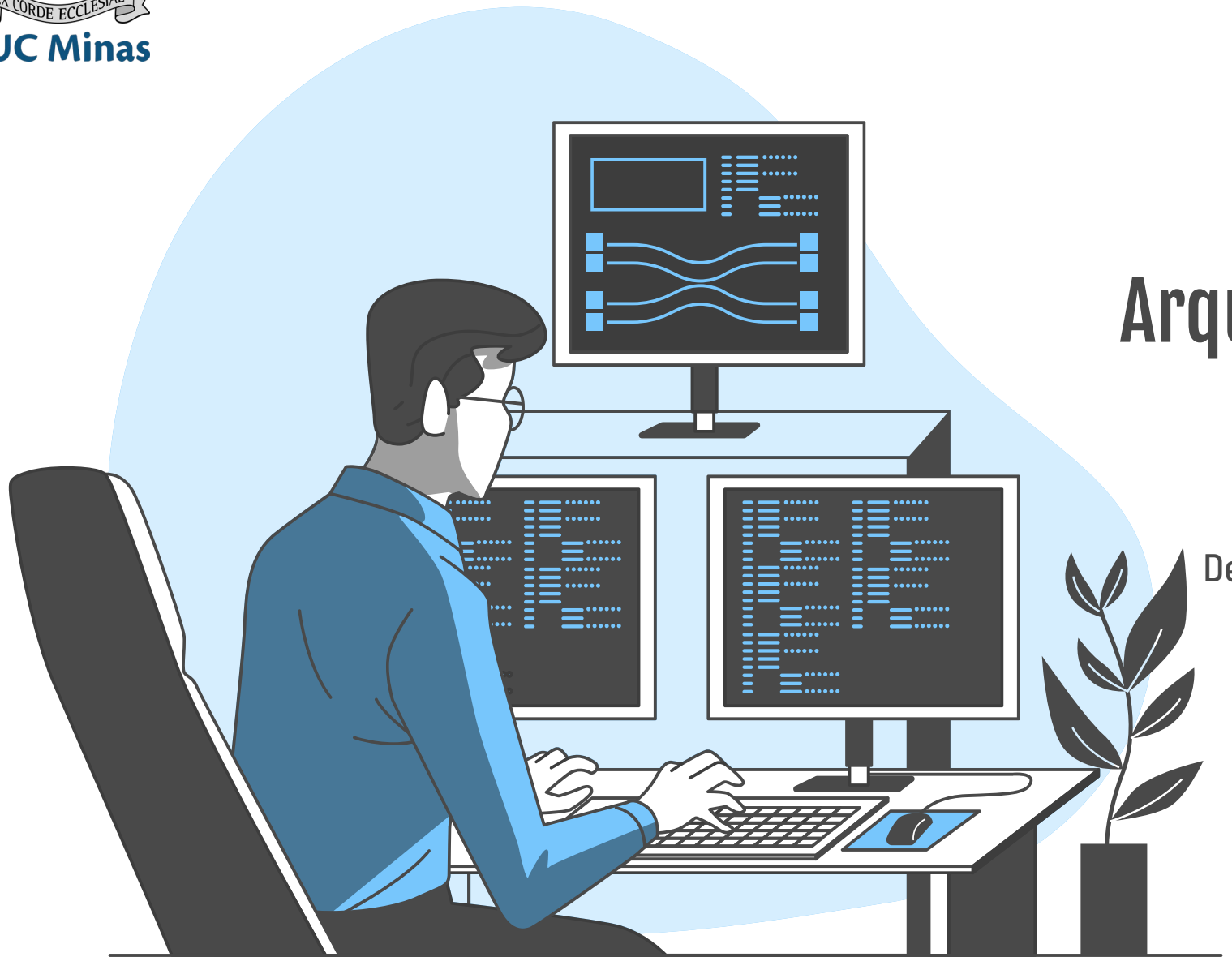
PUC Minas



Aula 04 · Back-end

# Arquitetura em camadas: os pacotes do projeto

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

# Nesta aula você vai

1

Entender o caminho de uma requisição pelas camadas

2

Saber o que vai em cada pacote: `controller`, `service`, `model`, `repository`

3

Diferenciar **DTO** de **entidade** e **DAO** de **repository**

4

Conhecer os pacotes de apoio: `config`, `security`, `exception`, `mapper`, `facade`, `enums`

5

Aplicar injeção de dependência pelo construtor

# Antes de começar

- Aula 02: projeto criado
- Aula 03: application.properties

Duração sugerida

**100 min**

**Código da disciplina**

Projetos de referência na pasta PROJETOS do GitHub

# Roteiro

01 Por que separar em camadas

02 Resumo de cada pacote

03 model: a entidade

04 repository: acesso ao banco sem SQL

05 dao: quando escrever o acesso à mão

06 dto: o que entra e o que sai da API

07 mapper: entidade ⇄ DTO

08 service e facade

09 Injeção de dependência

10 config, security, exception e enums

11 Aprofundando: boas práticas de arquitetura

01

# Por que separar em camadas

# Por que separar em camadas

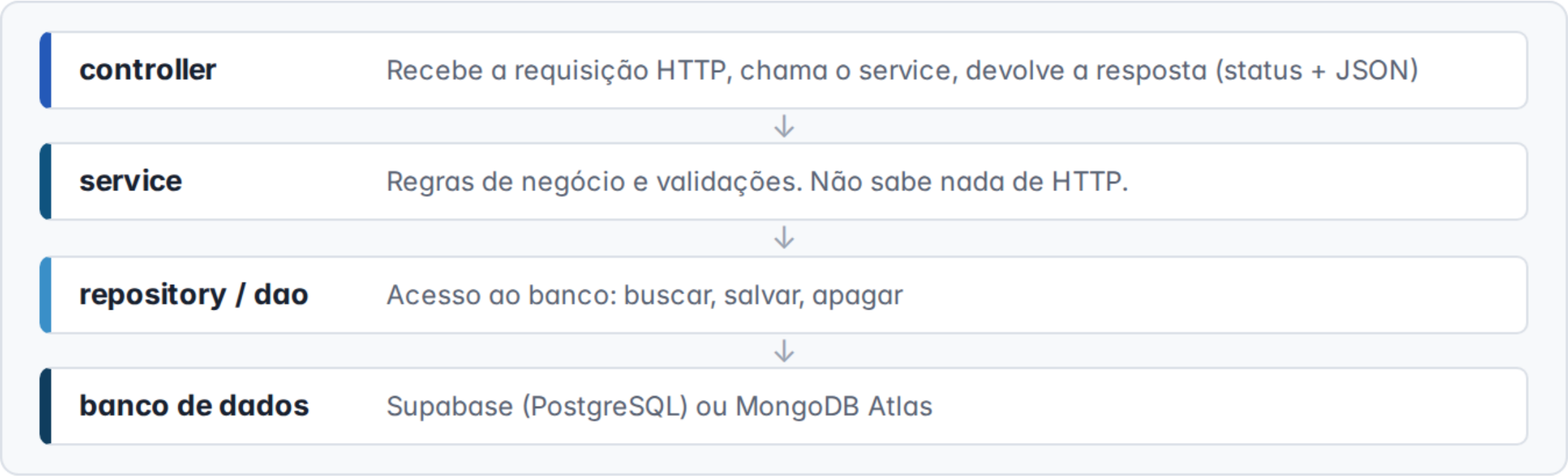
Seria possível escrever tudo dentro do controller: ler a requisição, validar, montar o SQL e devolver a resposta. Funciona para um exemplo de 20 linhas e vira um pesadelo com 20 endpoints. A arquitetura em camadas dá **uma responsabilidade para cada classe**: quando a regra de negócio muda, você mexe só no service; quando troca o banco, mexe só no repository.

01

**Por que separar em camadas**

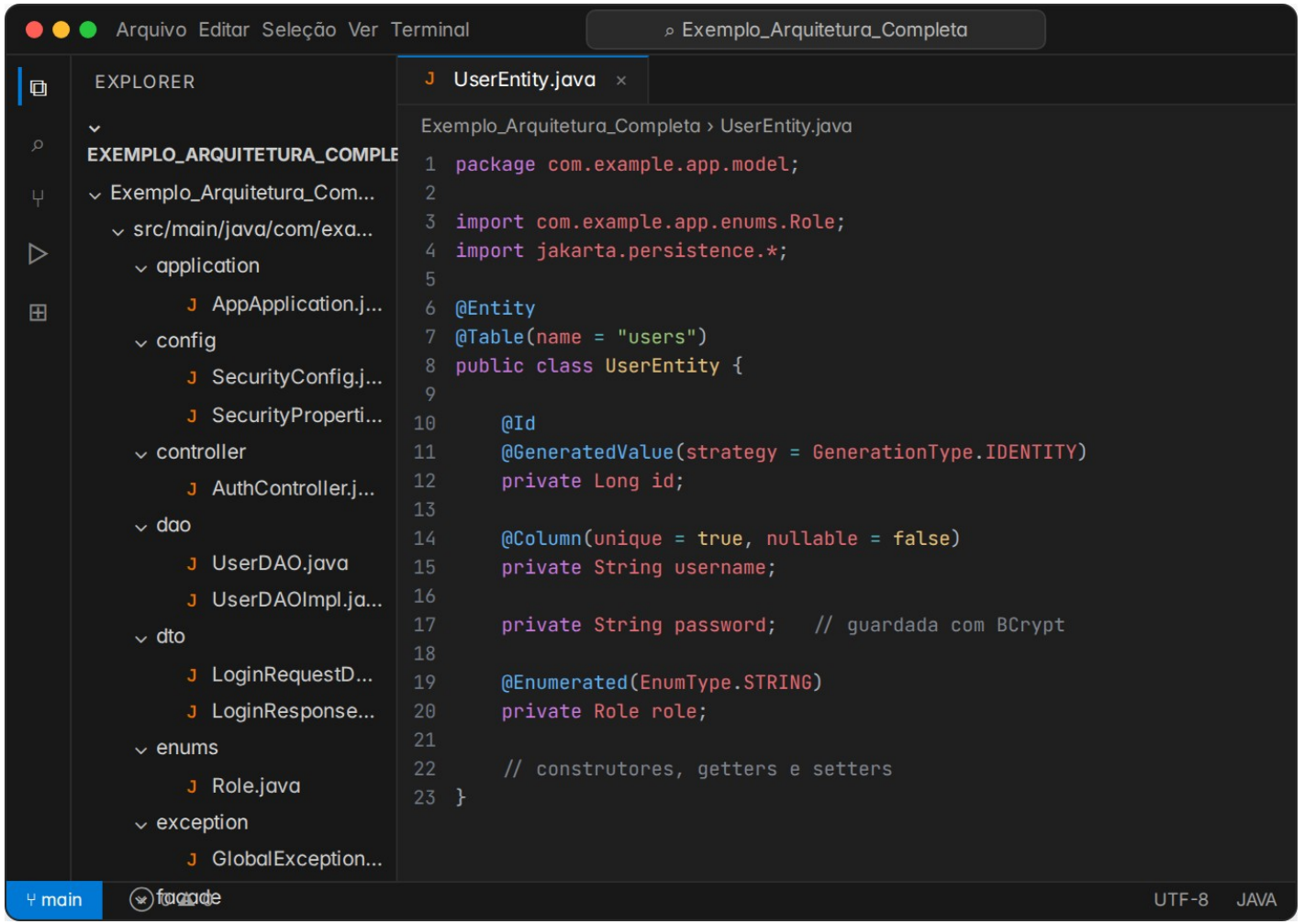
Aula 04 · Back-end

# Por que separar em camadas




O caminho de uma requisição. Cada camada só conhece a camada de baixo.

# Por que separar em camadas




Ao lado dessas camadas principais existem pacotes de apoio. A estrutura abaixo é a do projeto `Exemplo_Arquitetura_Completa`, que reúne todos eles:

Estrutura completa de pacotes com a entidade `UserEntity` aberta.

# 02

# Resumo de cada pacote

# Resumo de cada pacote

| Pacote | Responsabilidade | Anotações típicas |
|---|---|---|
| `application` | Classe com o `main` | `@SpringBootApplication` |
| `controller` | Endpoints HTTP | `@RestController`, `@GetMapping`, `@PostMapping` |
| `service` | Regras de negócio | `@Service`, `@Transactional` |
| `model` (ou `entity`) | Classes que viram tabela/documento | `@Entity`, `@Document` |
| `repository` | Interface de acesso ao banco | `extends JpaRepository` / `MongoRepository` |
| `dao` | Camada de acesso a dados escrita à mão | `@Component`, `@Repository` |
| `dto` | Objetos de entrada e saída da API | `record`, `@NotBlank` |

# Resumo de cada pacote (cont.)

| Pacote | Responsabilidade | Anotações típicas |
|---|---|---|
| `mapper` | Converte entidade ⇄ DTO | `@Component` |
| `facade` | Orquestra vários services para um caso de uso | `@Component` |
| `config` | Configurações e leitura de propriedades | `@Configuration`, `@Bean` |
| `security` | JWT, filtros de autenticação | `OncePerRequestFilter` |
| `exception` | Exceções próprias e tratador global | `@RestControllerAdvice` |
| `enums` | Valores fixos (papéis, status) | `enum` |

# 03

# model: a entidade

# model: a entidade

A **entidade** representa uma linha de uma tabela (JPA, Supabase) ou um documento de uma coleção (MongoDB). Ela tem os mesmos campos do banco, inclusive os que **não podem sair** para o cliente, como a senha.

03

**model: a entidade**

Aula 04 · Back-end

# 04

# repository: acesso ao banco sem SQL

# repository: acesso ao banco sem SQL

Com Spring Data você escreve **só a interface**. O Spring cria a implementação em tempo de execução, já com `findAll`, `findById`, `save`, `deleteById`, `count`... E gera consultas a partir do **nome do método**:

repository/UserRepository.java

```
public interface UserRepository extends JpaRepository<UserEntity, Long> {

    Optional<UserEntity> findByUsername(String username);            // WHERE username = ?

    List<UserEntity> findByRoleOrderByUsernameAsc(Role role);        // WHERE role = ? ORDER BY
username

    boolean existsByUsername(String username);                       // SELECT COUNT(*) > 0 ...

    @Query("select u from UserEntity u where u.username like %:texto%")
    List<UserEntity> buscar(@Param("texto") String texto);           // JPQL escrito à mão
}
```

# 05

# dao: quando escrever o acesso à mão

# dao: quando escrever o acesso à mão

**DAO** (*Data Access Object*) é um padrão mais antigo que o repository: uma interface + uma implementação que você escreve. Nos projetos ele aparece como uma camada **entre o service e o repository**, o que permite trocar a tecnologia do banco sem tocar no service, ou fazer consultas com `JdbcTemplate`/`MongoTemplate`.

**05**

**dao: quando escrever o acesso à mão**

Aula 04 · Back-end

# dao: quando escrever o acesso à mão

dao/UserDAO.java

```java
public interface UserDAO {
    Optional<UserEntity> findByUsername(String u);
    UserEntity save(UserEntity user);
}
```

dao/UserDAOImpl.java

```java
@Component
public class UserDAOImpl implements UserDAO {
    private final UserRepository repo;
    public UserDAOImpl(UserRepository repo) {
        this.repo = repo;
    }
    public Optional<UserEntity> findByUsername(String u) {
        return repo.findByUsername(u);
    }
    public UserEntity save(UserEntity user) {
        return repo.save(user);
    }
}
```

# dao: quando escrever o acesso à mão

i

## Observação

Em projetos pequenos, o service pode usar o repository direto, sem DAO. Use o DAO quando quiser isolar a tecnologia de banco ou tiver consultas complexas.

06

# dto: o que entra e o que sai da API

# dto: o que entra e o que sai da API

**DTO** (*Data Transfer Object*) é um objeto feito só para transportar dados entre o cliente e a API. Com ele você:

- **esconde** campos internos (a senha nunca vai na resposta);
- **valida** a entrada (`@NotBlank`, `@Email`);
- **desacopla** o formato da API do formato do banco.

Em Java moderno, DTOs são quase sempre `record`:

06

**dto: o que entra e o que sai da API**

Aula 04 · Back-end

# dto: o que entra e o que sai da API

dto/

```
public record LoginRequestDTO(
        @NotBlank String username,
        @NotBlank String password) { }

public record LoginResponseDTO(String token, List<String> roles) { }

public record AlunoResponseDTO(Long id, String nome, String email) { }   // sem senha
```

# 07

# mapper: entidade ⇄ DTO

# mapper: entidade ⇄ DTO

mapper/AlunoMapper.java

```java
@Component
public class AlunoMapper {
    public AlunoResponseDTO toDTO(Aluno a) {
        return new AlunoResponseDTO(a.getId(), a.getNome(), a.getEmail());
    }
    public Aluno toEntity(AlunoRequestDTO dto) {
        return new Aluno(dto.nome(), dto.email(), dto.curso());
    }
}
```

# 08

# service e facade

# service e facade

O **service** concentra a regra de negócio de **um assunto** (`UserService`, `AuthService`). A **facade** é opcional: ela atende **um caso de uso** que precisa de vários services, deixando o controller com uma linha só.

service/AlunoService.java

```java
@Service
public class AlunoService {
    private final AlunoRepository repo;

    public AlunoService(AlunoRepository repo) {
        this.repo = repo;
    }

    public Aluno inserir(Aluno a) {
        if (a.getNome() == null || a.getNome().isBlank())
            throw new DadosInvalidosException(
                "O nome do aluno é obrigatório.");
        return repo.save(a);
    }
}
```

facade/AuthFacade.java

```java
@Component
public class AuthFacade {
    private final AuthenticationManager auth;
    private final AuthService authService;
    private final AuthMapper mapper;
    // construtor...

    public LoginResponseDTO login(LoginRequestDTO r) {
        Authentication a = auth.authenticate(
          new UsernamePasswordAuthenticationToken(
              r.username(), r.password()));
        String token = authService.gerarToken(a);
        return mapper.toResponse(token, a);
    }
}
```

09

# Injeção de dependência

# Injeção de dependência

Nenhuma classe faz `new AlunoService()`. O Spring cria **uma instância** de cada classe anotada (`@Service`, `@Component`, `@RestController`…), chamada de **bean**, e entrega para quem pedir no **construtor**:

Injeção pelo construtor (forma recomendada)

```
@RestController
@RequestMapping("/api/alunos")
public class AlunoController {

    private final AlunoService alunoService;       // final: nunca muda

    public AlunoController(AlunoService alunoService) {   // o Spring injeta aqui
        this.alunoService = alunoService;
    }
}
```

# Injeção de dependência

✓ **Dica**

Você verá `@Autowired private UserService service;` em projetos antigos. Funciona, mas a injeção pelo construtor deixa o atributo `final`, facilita testes com Mockito e deixa explícito do que a classe depende.

# 10

# config, security, exception e enums

# config, security, exception e enums

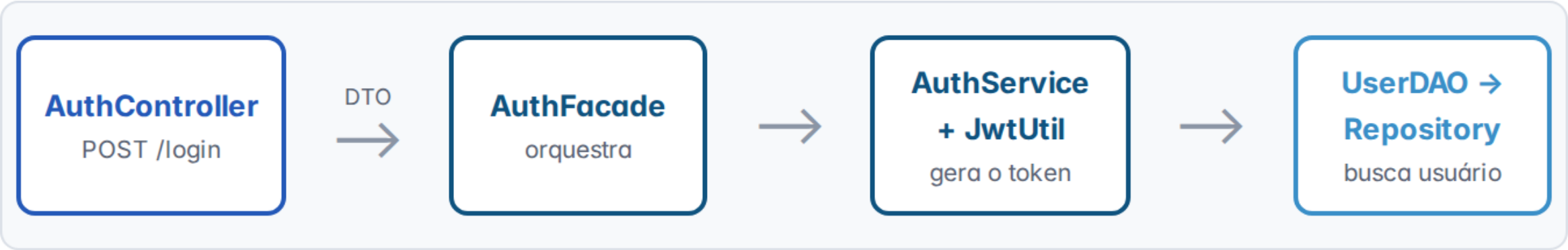
- **config**: classes `@Configuration` que criam beans (`SecurityConfig`, `CorsConfig`, `MongoConfig`) e classes `@ConfigurationProperties` que leem o `application.properties`.
- **security**: o que é específico de autenticação, como `JwtUtil` (gera e valida tokens) e o filtro `JwtAuthFilter` (aula 10).
- **exception**: exceções próprias (`AlunoNaoEncontradoException`) e o `GlobalExceptionHandler`, que converte cada exceção no código HTTP certo (aula 05).
- **enums**: valores fixos, como `enum Role { USER, ADMIN }`.

**10**

**config, security, exception e enums**

Aula 04 · Back-end

# config, security, exception e enums




Um login atravessando as camadas do `Exemplo_Arquitetura_Completa`.

11

# Aprofundando: boas práticas de arquitetura

# SOLID aplicado às camadas

## S – Responsabilidade única

Controller só fala HTTP; service só regra; repository só banco.

## O – Aberto/fechado

Nova regra de desconto? Uma nova classe, sem mexer nas existentes.

## L – Substituição

Qualquer implementação de `UserDAO` funciona no lugar da outra.

## I – Interfaces enxutas

Interfaces pequenas (`UserDAO`) em vez de uma interface gigante.

## D – Inversão de dependência

O service depende da **interface** do DAO, não da implementação.

# Controller gordo x controller enxuto

## Controller gordo

- Valida campos, monta SQL, trata erro com try/catch
- Difícil de testar
- Regra duplicada em vários endpoints

vs

## Controller enxuto

- Uma linha por endpoint: chama o service
- Regras testáveis com JUnit puro
- Erros tratados no `GlobalExceptionHandler`

# Aprofundando: boas práticas de arquitetura

Antes e depois

```java
// ruim: regra de negócio dentro do controller
@PostMapping
public ResponseEntity<?> criar(@RequestBody Aluno a) {
    if (a.getEmail() == null || !a.getEmail().contains("@"))
        return ResponseEntity.badRequest().body("email inválido");
    if (repo.existsByEmail(a.getEmail()))
        return ResponseEntity.status(409).body("já existe");
    return ResponseEntity.ok(repo.save(a));
}

// bom: o controller delega, o service decide
@PostMapping
public ResponseEntity<Aluno> criar(@Valid @RequestBody AlunoRequestDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(service.inserir(dto));
}
```

# Optional: sem NullPointerException

service/AlunoService.java

```java
// o repository devolve Optional<Aluno>
public Aluno buscar(String id) {
    return repo.findById(id)
               .orElseThrow(() -> new AlunoNaoEncontradoException(id));
}

public String nomeOuPadrao(String id) {
    return repo.findById(id).map(Aluno::getNome).orElse("(sem nome)");
}
```

# Aprofundando: boas práticas de arquitetura

`@Transactional` no service garante que várias operações no banco sejam **tudo ou nada**: se uma falhar, as anteriores são desfeitas. Use em métodos que gravam mais de uma coisa, como transferir saldo ou criar pedido com itens.

11

**Aprofundando: boas práticas de arquitetura**

Aula 04 · Back-end

# Pacote por camada x pacote por funcionalidade

## Por camada (usado na disciplina)

- `controller/`, `service/`, `repository/`
- Fácil de aprender e de achar cada tipo
- Em projetos grandes, cada pasta fica enorme

vs

## Por funcionalidade

- `aluno/`, `curso/`, `auth/`
- Cada pasta tem controller, service e repository do assunto
- Comum em sistemas grandes e microsserviços

# Aprofundando: boas práticas de arquitetura

| Camada | Como testar | Ferramenta |
|---|---|---|
| service | Teste unitário com o repository simulado | JUnit 5 + Mockito |
| controller | Requisição simulada, sem subir servidor | `MockMvc` (`@WebMvcTest`) |
| repository | Banco real ou em memória | `@DataJpaTest`, `@DataMongoTest` |
| tudo junto | Aplicação completa | `@SpringBootTest` |

# Aprofundando: boas práticas de arquitetura

src/test/java/.../AlunoServiceTest.java

```java
@ExtendWith(MockitoExtension.class)
class AlunoServiceTest {
    @Mock AlunoRepository repo;
    @InjectMocks AlunoService service;

    @Test
    void naoDeveInserirAlunoSemNome() {
        var aluno = new Aluno("", "ana@email.com", "ES");
        assertThrows(DadosInvalidosException.class, () -> service.inserir(aluno));
        verify(repo, never()).save(any());
    }
}
```

# Aprofundando: boas práticas de arquitetura

## Na prática

O projeto `Tests_JUnit_Mockito` e as pastas `src/test` de `Exemplo_Integracao` e `AraCoffee` têm testes prontos para cada camada. Leia junto com esta aula: os testes mostram, na prática, o que cada camada promete fazer.

12

# Quiz de revisão

Quiz · questão 1 de 5

# Pergunta

## Onde deve ficar a regra “o e-mail não pode se repetir”?

A Controller

B Service

C Model

D application.properties

# Resposta

## Onde deve ficar a regra “o e-mail não pode se repetir”?

A Controller

✓ Service

C Model

D application.properties

**Por quê:** Regras de negócio ficam no service; o controller só trata HTTP.

# Pergunta

## Qual a principal função de um DTO?

A Substituir o banco

B Definir o contrato de entrada/saída da API e esconder campos internos

C Criar tabelas

D Configurar o CORS

Quiz · questão 2 de 5

# Resposta

## Qual a principal função de um DTO?

A Substituir o banco

✓ Definir o contrato de entrada/saída da API e esconder campos internos

C Criar tabelas

D Configurar o CORS

**Por quê:** Ex.: a resposta nunca inclui a senha da entidade.

# Pergunta

## O que o Spring Data gera a partir do nome `findByCursoOrderByNomeAsc`?

**A** Um controller

**B** Uma consulta filtrando por curso e ordenando por nome

**C** Um DTO

**D** Uma migração

# Resposta

## O que o Spring Data gera a partir do nome `findByCursoOrderByNomeAsc`?

A. Um controller

✓ Uma consulta filtrando por curso e ordenando por nome

C. Um DTO

D. Uma migração

**Por quê:** São os *query methods* derivados do nome.

# Pergunta

## Qual a forma recomendada de injeção de dependência?

**A** `new AlunoService()`

**B** Pelo construtor, com atributo `final`

**C** Variável estática

**D** @Autowired em método qualquer

# Resposta

## Qual a forma recomendada de injeção de dependência?

A `new AlunoService()`

✓ Pelo construtor, com atributo `final`

C Variável estática

D @Autowired em método qualquer

**Por quê:** Deixa as dependências explícitas e facilita testes com Mockito.

# Pergunta

## O que uma *facade* faz?

A Acessa o banco diretamente

B Orquestra vários services em um caso de uso

C Substitui o controller

D Converte JSON

Quiz · questão 5 de 5

# Resposta

## O que uma *facade* faz?

A Acessa o banco diretamente

✓ Orquestra vários services em um caso de uso

C Substitui o controller

D Converte JSON

**Por quê:** Ex.: o login usa AuthenticationManager, AuthService e AuthMapper.

Fechamento

# Resumo da aula

- ✓ Cada camada tem uma responsabilidade: **controller** (HTTP), **service** (regras), **repository/dao** (banco).
- ✓ **Entidade** espelha o banco; **DTO** é o contrato da API; o **mapper** converte um no outro.
- ✓ O **repository** do Spring Data gera consultas pelo nome do método; o **DAO** é uma camada escrita à mão.
- ✓ A **facade** junta vários services em um caso de uso.
- ✓ O Spring injeta dependências pelo **construtor**; ninguém usa `new` em beans.

# Exercícios

1. Desenhe o caminho de um `DELETE /api/alunos/5` pelas camadas.

2. Crie `AlunoRequestDTO` e `AlunoResponseDTO` como `record` e um `AlunoMapper`.

3. Escreva três métodos derivados no repository: buscar por curso, contar por curso, existir por e-mail.

4. Refatore um controller que usa `@Autowired` em atributo para injeção pelo construtor.

# Para ir além

- Projeto `PROJETOS/SpringBoot/Exemplo_Arquitetura_Completa`
- docs.spring.io/spring-data/jpa/reference: query methods

**Repositório da disciplina**

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

*Links e versões citados podem ter mudado. Confira sempre a documentação oficial.*




# Obrigado!

Dúvidas sobre a aula 04?

**Prof. Dr. João Paulo Aramuni**
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

**GitHub da disciplina**
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web